Second Progress Report

Freely distributable tools for finding web vulnerabilities

Tomáš Meravý Murárik

Architecture of the experiment

- Objective:
  - Define a structured and repeatable methodology for evaluating open-source web vulnerability scanners against selected CWEs (CWE-89, CWE-552, CWE-1104).
- Key Components:
  - Controlled testing environment
  - Carefully selected target applications
  - Two-phase testing methodology
  - Standardized metrics for comparison

Environment

- Testing Setup:
  - Isolated, controlled environment
  - Dedicated vulnerable targets:
    - Custom vulnerable web application
    - Laravel-based application
    - Damn Vulnerable Web Application (DVWA)
- Purpose:
  - Ensure safe testing
  - Avoid false positives caused by external network conditions
  - Guarantee reproducibility

Toolset

- Tools Selected:
  - Dirsearch – Specialized hidden file & directory enumeration
  - w3af – Full-featured web vulnerability scanner (extensible plug-in architecture)
  - Nikto – High-performance server scanner with large signature database
  - Nuclei – One of the most popular open source projects of this sort
  - Wapiti – Black-box scanner focusing on injection and file disclosure flaws
  - Zap – GUI-based scanner developed by owasp
  - Grabber – Lightweight scanner for small web applications
- Rationale:
  - Covers general-purpose + specialized tool
  - Enables comparative analysis of scanning depth, accuracy, and false-positive rates

Testing Approach

- Phase 1 — Default Configuration Tests
  - Use tools with factory settings
  - Establish baseline detection capabilities
  - Identify usability differences and noise levels
- Phase 2 — Optimized Configuration Tests
  - Apply targeted parameters (e.g., deeper recursion, aggressive checks, specific injection modules)
  - Improve precision for selected CWEs
  - These results provide the main data for evaluation

Execution Workflow

- Initialize test environment
- Deploy target application
- Run Phase 1 scans on all tools
- Record detection results and tool behavior
- Tune tool configurations
- Run Phase 2 scans
- Collect metrics (Detection accuracy)
- Compare general-purpose vs. specialized tools

Preliminary Results

- Default-settings testing
- Custom vulnerable web application
- Focus: Hidden file & directory discovery
- Multiple intentionally placed files:
  - Easy-to-discover: .env, admin/, config/, *.log
  - Hard-to-discover: dbx309183vbg$.out, quiyana.log, userLog321393029gx.txt

Detection Summary

- Nuclei — 0 results
- Wapiti — 0 potentially dangerous files
- Nikto — config/, admin/, .git
- OWASP ZAP — Same as Nikto + resource folder
- Dirsearch — .env, admin/, config/, .git/, db.log, error/

Evaluation

- Dirsearch significantly outperformed general-purpose scanners
  - Found .env, db.log, multiple directories, and error pages
  - Shows strength of specialized enumeration tools
- General-purpose scanners (ZAP, Nikto, Wapiti) focus more on vulnerabilities, not deep file enumeration
- Nuclei, without tailored templates, is ineffective for hidden file scanning
- Hard-to-detect files require extended wordlists or brute-force mode

Plan for Completing the Project

- Complete full evaluation of selected tools across the three CWE focus areas:
  - CWE-89: SQL Injection
  - CWE-552: Exposure of Files/Directories
  - CWE-1104: Use of Unmaintained Third-Party Components
- The next phase of the project expands on preliminary hidden-file testing and moves into complete vulnerability assessment and comparison

Upcoming Experimental Work

- Extend Experiments to All CWEs
  - Execute SQL injection tests
  - Test outdated/unmaintained component detection
  - Re-run hidden file detection with optimized configurations
- Run Two-Phase Testing for Every CWE
  - Phase 1: Default settings
  - Phase 2: Tuned/optimized configurations for each tool

Questions?

Thank you for you attention